



Shared CPU-FPGA Filesystem Access on fast NVMe Devices

Master's Thesis

Author: Bc. VLADISLAV VÁLEK
Advisor: Prof. Dr. DIRK KOCH
Opponent: Prof. Dr. HOLGER FRÖNING

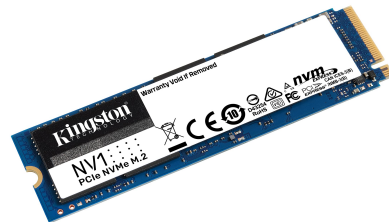
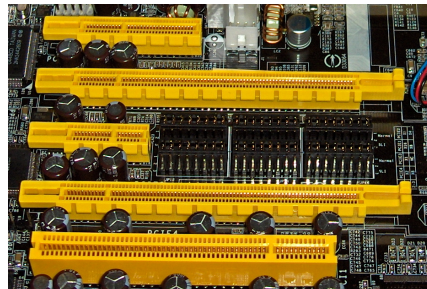
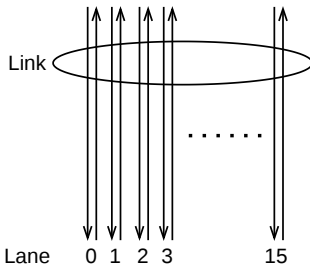
Heidelberg, 30-March-2026

- Motivation and goals
- PCI Express and NVMe essentials
- Hardware architecture
- Shared filesystem access
- Evaluation highlights
- Demo
- Conclusion

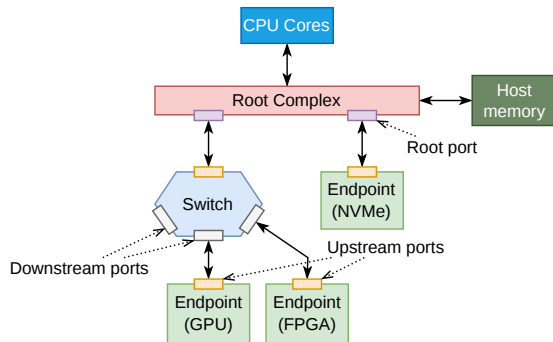
- Conventional CPU-facilitated workflow introduces an overhead
- Host memory used for staging (double copying)
- Direct access to NVMe storage from accelerator
- Standardized filesystem

Goal of this work: enable shared CPU-FPGA filesystem access to NVMe while preserving a standard Linux workflow.

- Since 2003
- Streaming & memory-mapped
- Serial-parallel hybrid
- Specification – Gen7 (242 GBps)
- Devices – Gen5 (63.015 GBps)
- Support on FPGAs



- Switched point-to-point interconnect
- Tree topology
- Root Complex, Switches, Endpoints
- Address-based routing*
- Peer-to-peer transport (host bypassing)
- Transaction Layer Packet (TLP)
 - Memory Write request
 - Memory Read request
 - Completion



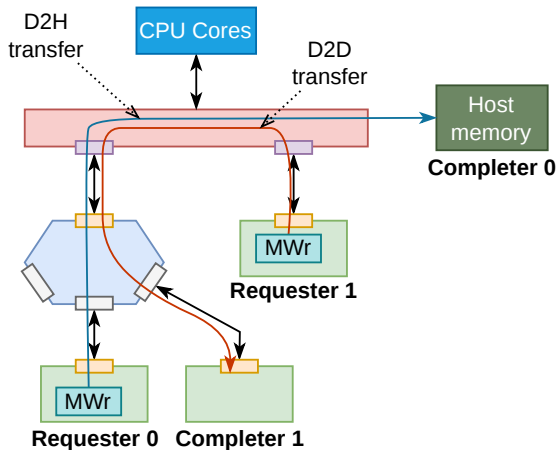


Fig.: Write request path

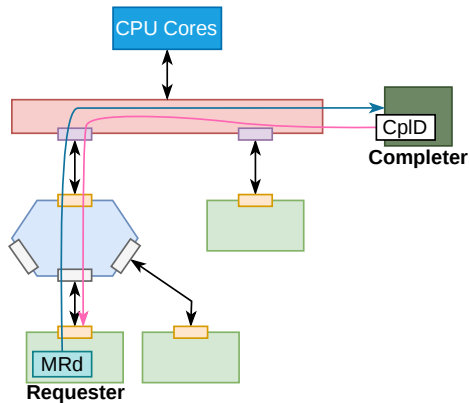
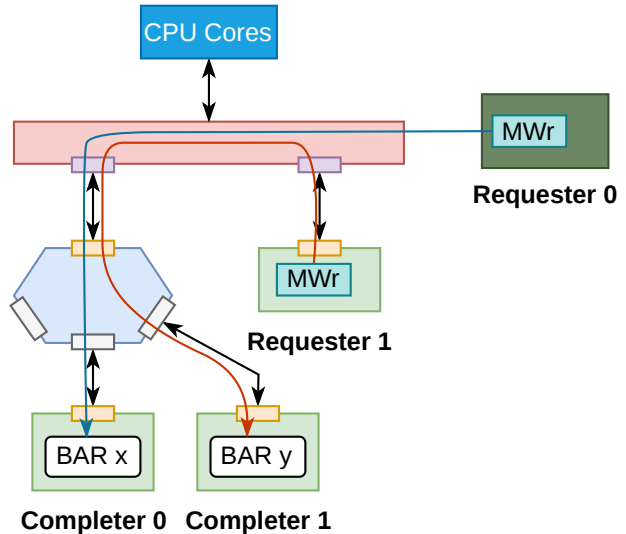


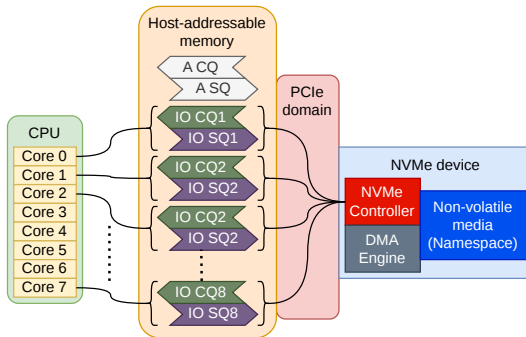
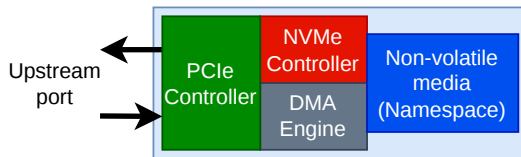
Fig.: Read request path

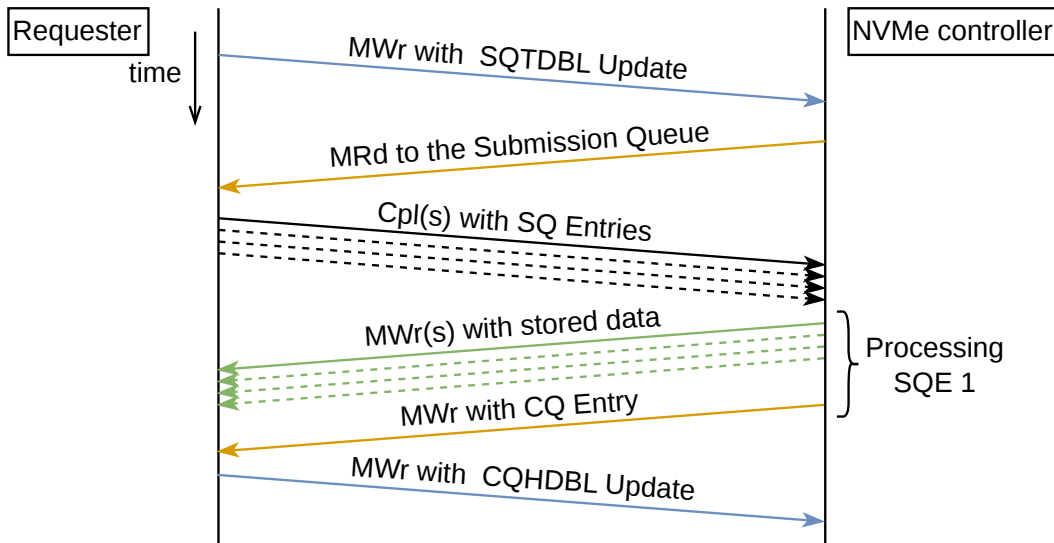
- Configured on startup
- Bus Mastering
- Base Address Register (BAR)
- Enable memory-mapped IO

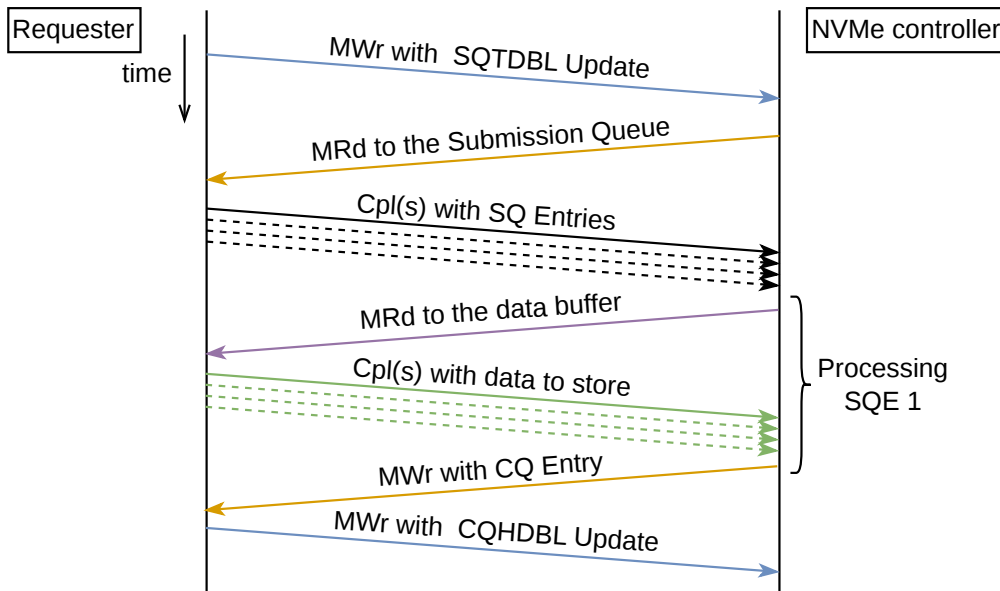


Non-volatile Memory Express (NVMe)

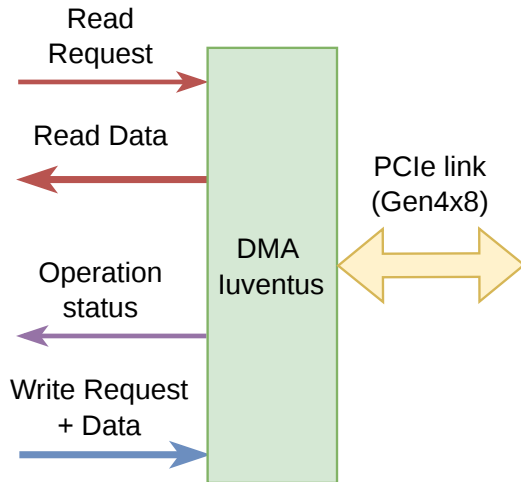
- Command based
- Queue-pair model: submission and completion queues
- Up to 2^{16} queues
- Up to 2^{16} commands
- Pointers to synchronize queue state (doorbells)
- Namespace divided to sectors
- Logical Block Address (LBA)

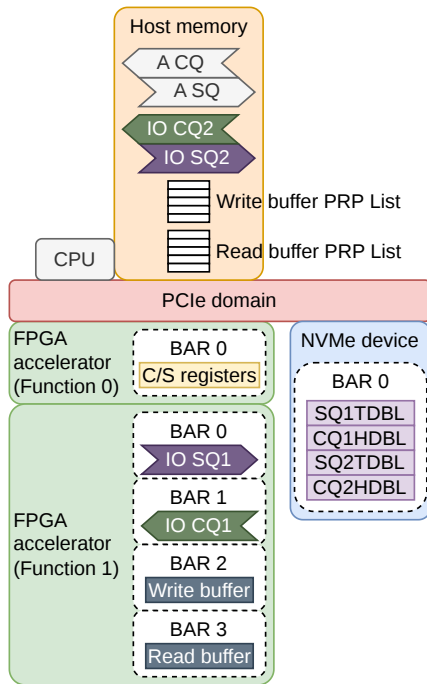




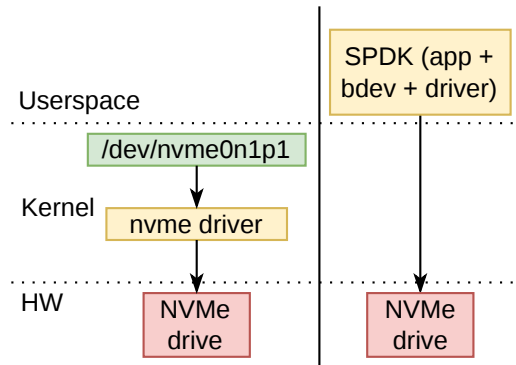


- FPGA card: AMD Alveo U55C
- Up to PCIe Gen4x16
- Accelerator BARs hold queues and data buffers
- DMA luventus IP
- Interface to user logic
- Maximum IO size of 256 LBAs

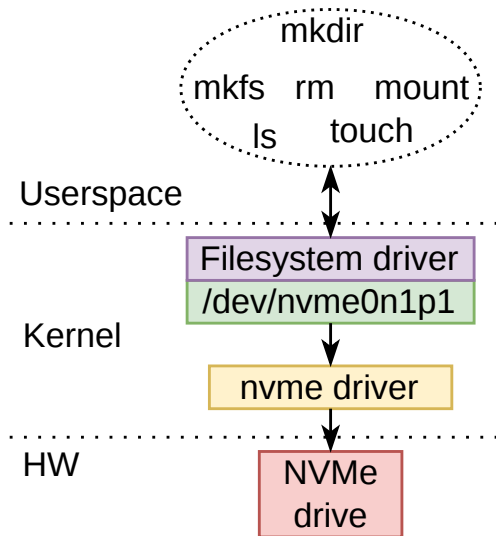




- System calls are expensive
- Kernel development difficult
- More libraries
- Easier debugging
- Storage Performance Development Kit (SPDK)
- Poll mode driver (PMD)
- No interrupts
- Lower latency, higher throughput



- Everything in kernel
- Block vs. character devices
- Only bdevs run filesystem



1. Keep userspace drivers
2. Create block device
3. Publish bdev to the kernel
4. Initialize filesystem (exFAT)
5. Mount filesystem
6. Forward IO requests to the ublk server

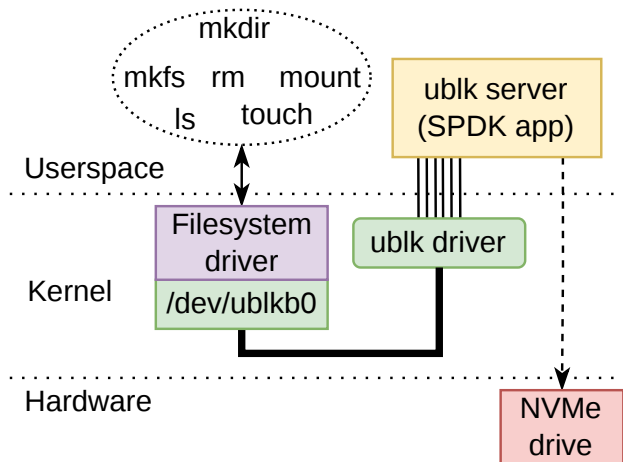
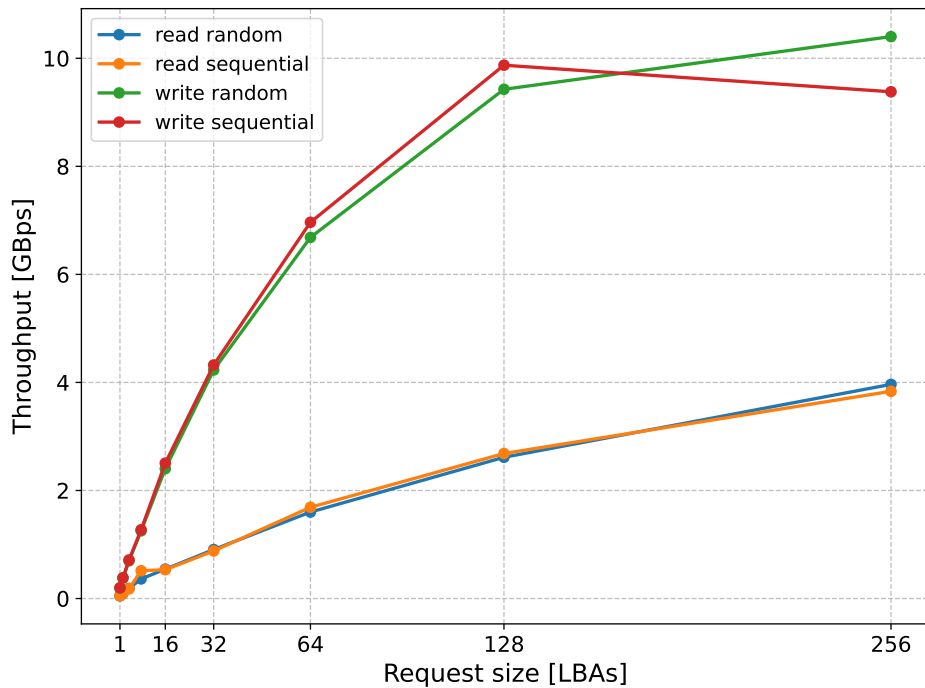
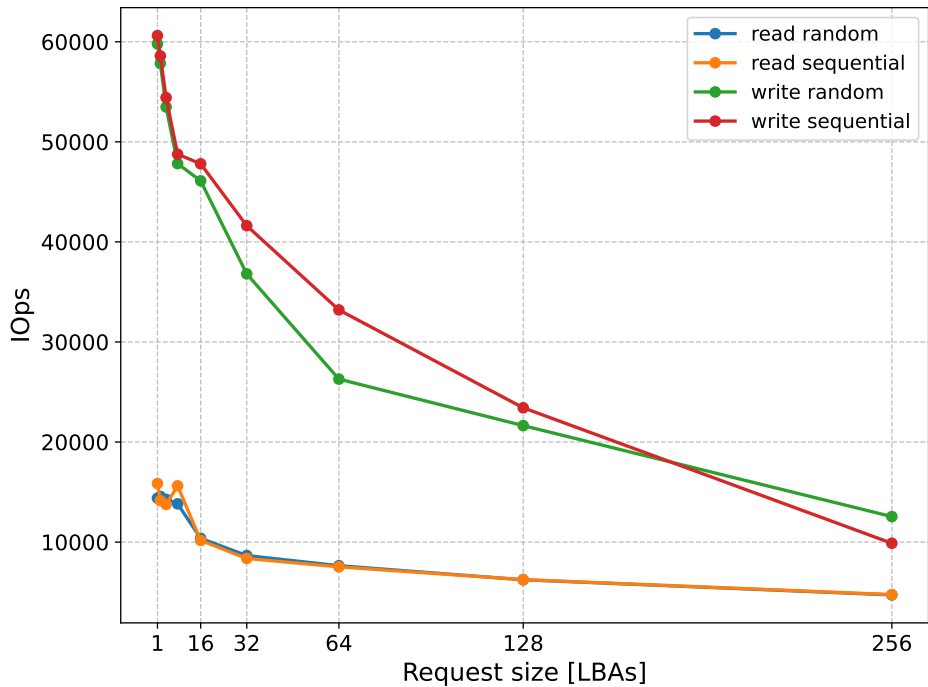


Table: FPGA resource consumption after design implementation phase (fully placed and routed)

Resource type	DMA luventus		PCIe IP		Test design total	
	abs	%	abs	%	abs	%
LUT	12532	0.96	9155	0.70	24792	1.90
LUTRAM	268	0.04	1148	0.19	1938	0.32
FF	22208	0.85	24903	0.96	52498	2.01
BRAM Tile	134.5	6.67	30.5	1.51	194.5	9.65
URAM	0	0	0	0	11	1.15

- Measured over period of 2^{28} cycles (~ 1 second)





- Timestamp width 28 bits (~1 second delay)
- Results stored in histogram (1 bin = 512 ns)
- 50000 repetitions

Op.	Access	Size [LBAs]	Min	P_{50}	P_{99}	Max
RD	Random	1	46.64	66.71	81.75	1952.72
		256	143.27	203.46	238.26	357.50
	Sequential	1	62.12	63.15	65.97	173.30
		256	144.12	201.50	237.61	1904.40
WR	Random	1	8.34	8.61	107.28	5379.27
		256	54.05	62.65	454.14	6692.82
	Sequential	1	8.34	8.45	8.70	10.19
		256	54.10	71.57	251.26	5860.76

Demo

Shared read/write filesystem access.

- PCIe peer-to-peer path avoids mandatory host-memory staging
- Linux filesystem access is maintained
- Design verified using functional verification (CocoTB)
- Validated in hardware
- Shared CPU-FPGA filesystem access over NVMe was demonstrated
- Future development
 - More outstanding commands
 - Support for more NVMe drives
 - Alveo V80 support
 - Support for both NVMe and host memory transfers

Thank you for your attention!

Any questions?

